

Sistemas de Informação

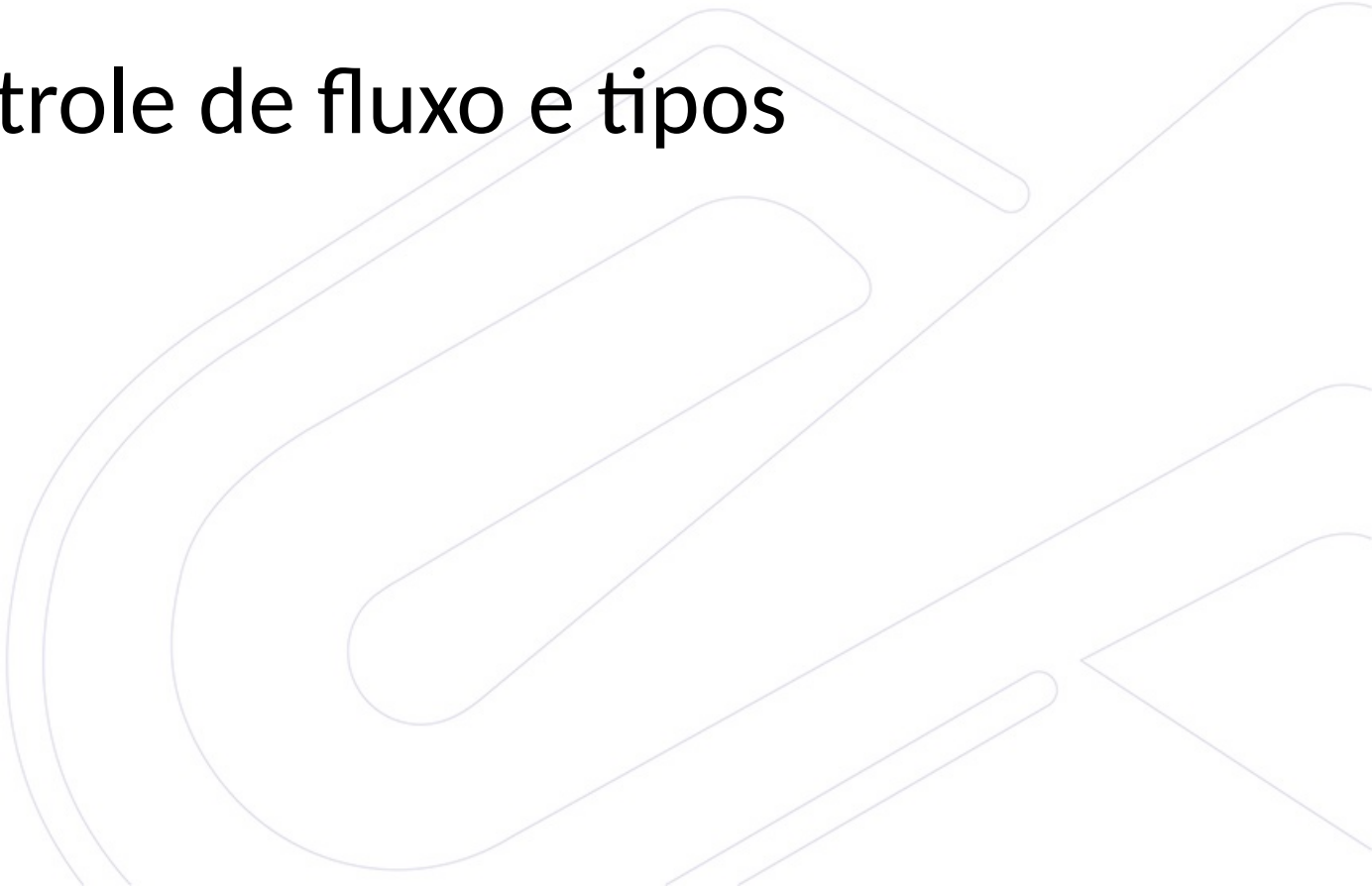


Disciplina: Programação I

Prof. Renê Rodrigues Veloso

Para fecharmos essa parte de introdução à linguagem Java

... mais sobre vetores, controle de fluxo e tipos numéricos.

A decorative graphic in the bottom right corner consisting of several overlapping, rounded rectangular outlines in a light blue color, creating a modern, abstract design.

Vetores e Matrizes (*arrays*)

Arrays em Java são **objetos**!

Uso do operador ***new***

Os nomes dos objetos são apenas referências às regiões de memória alocadas:

```
int[] vet1 = {10, 20, 30, 40};  
int[] vet2;
```

```
vet2 = vet1;  
vet2[2] = 3;
```

```
for(int i: vet1)  
    System.out.println(i);
```

Saída:



```
10  
20  
3  
40
```

Vetores e Matrizes (*arrays*)

Arrays em Java são **objetos**!

Possuem atributos e métodos próprios: *length*, *clone*, ...

Outros métodos para manipular arrays está na classe **Java.util.Arrays**

```
int[] vet1 = {10, 20, 30, 40};
int[] vet2;

vet2 = vet1.clone();
vet2[2] = 3;

for(int i: vet1)
    System.out.println(i);

for(int i=0; i < vet2.length; i++)
    System.out.println(vet2[i]);
```

Saída:

```
10
20
30
40
10
20
3
40
```

Vetores e Matrizes (*arrays*)

For-each (para cada)

Sintaxe:

```
for (type variable : arrayname) {  
    ...  
}
```

```
String[] carros = {"Volvo", "BMW", "Ford", "Fiat"};
```

```
for (String i : carros) {  
    System.out.println(i);  
}
```

Saída:

```
Volvo  
BMW  
Ford  
Fiat
```


Vetores e Matrizes (*arrays*)

Arrays multidimensionais

```
int[][] myNumbers = { {1, 2, 3, 4}, {5, 6, 7} };
```

```
for (int i = 0; i < myNumbers.length; ++i) {  
    for(int j = 0; j < myNumbers[i].length; ++j) {  
        System.out.println(myNumbers[i][j]);  
    }  
}
```

Saída:

1
2
3
4
5
6
7

Operador *instanceof*

Compara um objeto com um tipo específico. Pode ser usado para testar se um objeto é uma instância de uma classe específica, uma instância de uma subclasse ou se implementa uma interface específica.

Saída:

```
entrada é um Scanner? true
```

```
Scanner entrada = new Scanner(System.in);
```

```
System.out.println("entrada é um Scanner? "+(entrada instanceof Scanner));
```

Java Wrapper Classes

Classes que “empacotam” tipos primitivos e nos oferecem mais recursos.

Os tipos primitivos são automaticamente convertidos para um objeto e vice-versa, quando requerido por uma função ou escopo do programa.

chamado de Autoboxing e Unboxing.

Classes:

String

StringBuilder

Numbers:



Java Wrapper Classes: números e outros

Integer

Byte

Double

Short

Float

Long

Boolean

Character



Strings

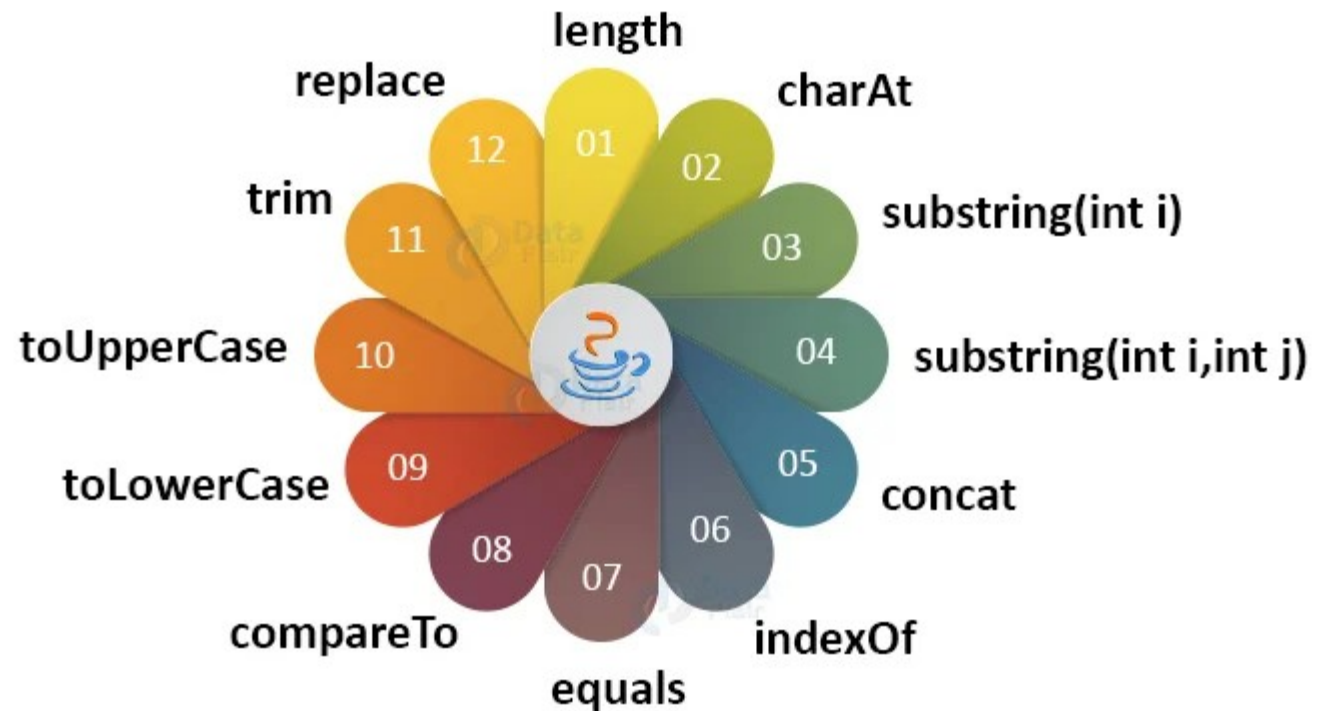
Strings, em java, são imutáveis!

```
String a = "abcd";  
a[2] = 'f';  
System.out.println(a);
```



error: array required, but String found
a[2] = 'f';

String Methods in Java



Strings

Lendo uma string do teclado;

```
Scanner in = new Scanner(System.in);  
String nome;
```

```
nome = in.nextLine();  
System.out.println(nome);
```



StringBuilder

permite criar e manipular strings dinamicamente

Armazena caracteres especificados pela sua capacidade, caso ela seja excedida, é aumentada para acomodar os caracteres adicionais;

Não precisa alocar novos objetos quando realiza uma concatenação;

```
StringBuilder buffer = new StringBuilder("Rene Veloso");
```

```
buffer.append(" - Unimontes");  
System.out.println("oi "+buffer);  
System.out.println(buffer.reverse());
```

Saída

```
oi Rene Veloso - Unimontes  
setnomiU - osoleV eneR
```